CS528 Homework 2: Web Graph Analysis

GitHub: ThomasFYD
September 27, 2026

1. Project and data setup

The program analyzes 12,000 HTML pages using custom Python code. It builds a directed graph and calculates incoming/outgoing degree statistics, PageRank, and closeness centrality. It is single-threaded and does not use graph libraries.

Project: **CS-528-YF**
Project ID: **dazzling-seat-508219-j3**
Bucket: **cs528-hw2-yf-2026**, prefix: **data/**
Repository: https://github.com/ThomasFYD/CS528-HWs

The bucket uses Standard storage in us-central1, uniform bucket-level access, and public access prevention is not enforced. The allUsers principal was granted roles/storage.objectViewer at the bucket level, allowing anonymous listing and reading but not writing. No per-object ACLs are needed. The bucket remains available for grading.

The instructor's generator uses seed 0. The following commands generate the dataset in data/ and upload it. Setup has already been completed; these commands are not needed to run the analysis.

```
mkdir -p data
cd data
python ../generate-content.py -n 12000 -m 325
cd ..
gcloud storage cp data/*.html gs://cs528-hw2-yf-2026/data/
gcloud storage buckets add-iam-policy-binding \
  gs://cs528-hw2-yf-2026 \
  --member=allUsers --role=roles/storage.objectViewer
```

A listing confirmed 12,000 files. An anonymous request to https://storage.googleapis.com/cs528-hw2-yf-2026/data/0.html returned HTTP 200. Because of exclusive loop bounds in the supplied generator, -m 325 produces 0 to 323 actual links per page.

2. Running the code

Use Python 3.12. No Google credentials are needed for the public dataset. From a new directory:

```
git clone https://github.com/ThomasFYD/CS528-HWs.git
cd CS528-HWs
git checkout 78848ab
cd CS528-HW2
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pytest -v
python -u -m src.main
```

Commit 78848ab identifies the report's source and log snapshot. Run commands from CS528-HW2. The bucket, prefix, and expected file count are constants in src/main.py.

3. Algorithms and recovery

Graph and degree statistics. HTMLParser extracts link targets, and outgoing/incoming adjacency lists store the graph. Targets outside the dataset are ignored. Repeated links are counted separately for degrees and PageRank. Mean, median, minimum, maximum, and quintile boundaries are computed from the degree lists. Percentiles use linear interpolation at sorted position (N - 1)q.

PageRank. Every node starts at 1/N. Synchronous updates use damping 0.85. If D is the current rank mass of nodes with no outgoing links, the update is:

```
newPR(v) = 0.15/N + 0.85*D/N
           + 0.85*sum(PR(u)/outdegree(u) for each edge u -> v)
```

Dangling-node mass is redistributed uniformly. Iteration stops when the sum of absolute score changes is at most 0.005, with a maximum of 1,000 iterations.

Stopping-rule clarification: the assignment describes a 0.5% change in the sum of PageRanks. The code instead checks the L1 change of the rank vector. These are not identical: the rank sum stays approximately 1 under this implementation even when individual scores change. This is the rule used for the reported results; instructor/TA confirmation is still needed.

Closeness. Custom single-threaded BFS with integer bitsets follows outgoing links. If r other nodes are reachable and their distance sum is S, the score is (r/S) multiplied by r/(N - 1). It is zero when no other node is reachable. This is directed outward closeness with reachable-fraction normalization, not inward or undirected closeness. Duplicate edges do not affect shortest-path distances.

Recovery. Downloads use a fresh anonymous client per object, up to three attempts for selected transient failures, and object-generation pinning. A checkpoint is saved every 25 pages using a temporary file and replacement. A manifest of names and generations is checked before resuming, and incoming links are reconstructed after all pages are processed.

The default command performs a fresh bucket read. **--resume** re-lists objects and resumes a matching checkpoint, or starts fresh if none exists. **--use-cache** skips downloading by loading the completed graph cache; it is for development, not full-run timing. The options are mutually exclusive. Load only trusted pickle files generated by this program.

4. Tests

Six independent tests passed on the VM in 0.30 seconds using Python 3.12.3 and pytest 9.1.1. They use small fixed fixtures, not the generated 12,000-page graph:

1. A three-node PageRank graph is compared with known scores at tolerance 1e-12: A = 0.3877897117, B = 0.2148106275, C = 0.3973996608.
2. A dangling-node graph checks nonnegative scores, total rank 1, and the expected rank ordering.
3. A five-node bidirectional star checks closeness 1 for the center and 4/7 for each leaf.
4. A directed chain with an isolated node checks scores 4/9, 1/3, 0, and 0.
5. An interrupted download checks resumed progress and preservation of duplicate links.
6. A changed object generation checks rejection of an outdated checkpoint.

Tests are in tests/test_pagerank.py, tests/test_closeness.py, and tests/test_checkpoint_graph.py. The known-score PageRank test uses a tighter tolerance than the full dataset run.

5. Results

All three environments produced **12,000 nodes and 1,946,934 links**, with matching degree statistics, PageRank top five, and best closeness node.

Statistic	Outgoing	Incoming
Mean	162.2445	162.2445
Median	162.0	162.0
Minimum / maximum	0 / 323	115 / 233
0%, 20%, 40%	0, 65, 130	115, 152, 159
60%, 80%, 100%	195, 260, 323	165, 173, 233

Rank	Page	PageRank
1	5207.html	0.0001955251
2	7443.html	0.0001828610
3	7400.html	0.0001796367
4	950.html	0.0001744095
5	369.html	0.0001733209

PageRank stopped after 3 iterations; final L1 change was 0.0011577820 and the rank sum was 1.0000000000. The best closeness node was **10376.html**, with score **0.5046473483**.

6. Runtime comparison

Stage (seconds)	Mac	Cloud Shell*	VM
Graph preparation	1189.15	2165.93*	1532.41
Degree statistics	0.02	0.04	0.02
PageRank	0.28	1.10	0.60
Closeness	30.91	65.13	39.86
Total invocation	1220.50	2232.43*	1573.00

* Cloud Shell resumed from 10,375 completed pages in the last invocation. Its total is **not a full-run total**, although the final algorithms processed the complete graph. Five progress-bearing checkpoint intervals sum to 15,166.1 seconds; that partial sum excludes listing, idle gaps, unsaved work, earlier abandoned runs, and final analysis.

The laptop was a MacBook Pro with Python 3.12.0 in its recorded test log; CPU/RAM were not recorded. Cloud Shell used Python 3.12. The VM was e2-medium (2 vCPUs, 4 GB RAM), AMD Rome, us-central1-a, Ubuntu Minimal 24.04 LTS, and Python 3.12.3.

Timings use time.perf_counter(). Graph preparation accounts for about 97% of the full Mac and VM totals. Sequential downloads, connection setup, parsing, and checkpoint writes dominate that stage; repeated BFS makes closeness the largest analysis stage. Network conditions and CPU availability may explain differences, but they were not isolated. The Mac baseline predates the fresh-client and checkpoint changes, so the totals are not a controlled hardware-only comparison.

7. VM setup and cleanup

The e2-medium VM used a 10 GB balanced boot disk, ephemeral external IPv4, no scheduled backups, and no Ops Agent. The following packages were installed before running the installation commands on page 1:

```
sudo apt-get update
sudo apt-get install -y git python3-venv python3-pip tmux
```

The full run was launched inside tmux without --resume or --use-cache:

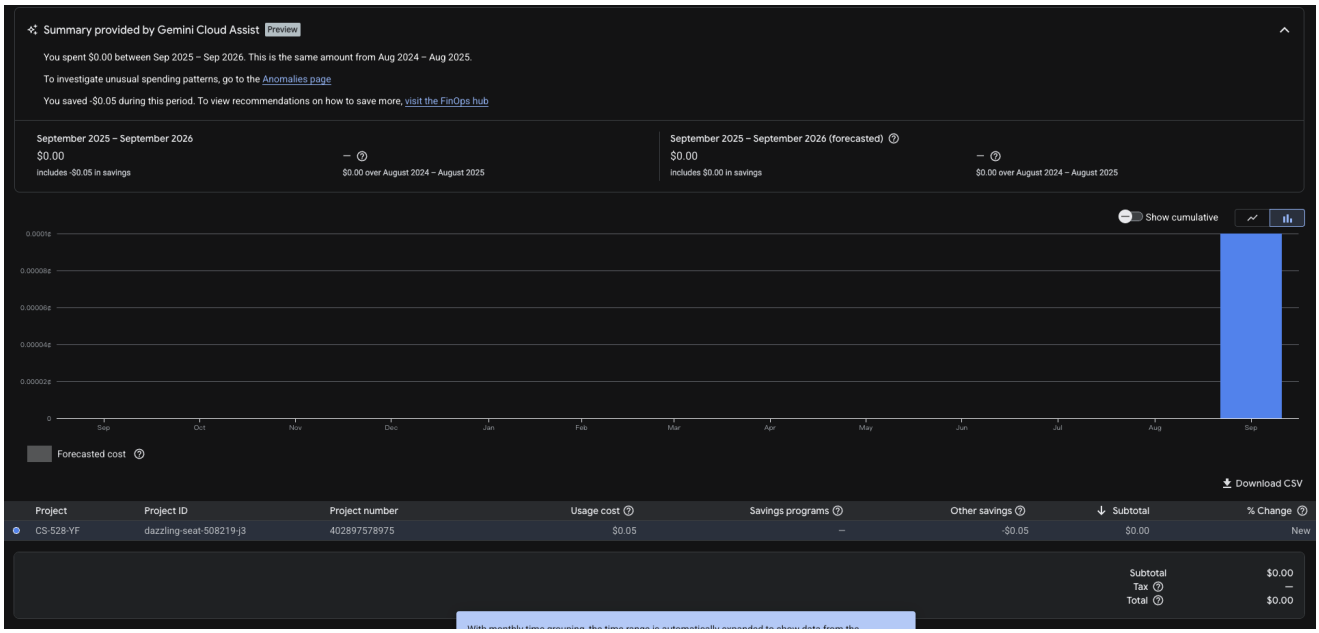
```
tmux new -s hw2
# In the session, activate the project virtual environment.
set -o pipefail
python -u -m src.main 2>&1 | tee results/vm_full_run.txt
```

The VM completed in 1,573 seconds (26 minutes 13 seconds). Its log was copied to the Mac and pushed to GitHub. The VM cs528-hw2-vm and its boot disk were then deleted; the console showed empty instance and disk lists. The public bucket was retained for grading.

8. Cloud spending

At the September 27, 2026, 00:07 America/New_York capture, the billing report showed **$0.05 usage cost, -$0.05 other savings, and $0.00 net total**. The screenshot covers September 2025-September 2026; the user reports project usage only during September 25-27, 2026.

These are provisional displayed amounts, not a finalized total. Recent VM charges may not yet be included. Google notes that cost reporting can take a day or longer than 24 hours. The retained bucket may continue to incur charges. The bill should be refreshed before final submission.



Cloud Billing Reports screenshot supplied by the user, September 27, 2026.

GitHub repository and full logs

GitHub: https://github.com/ThomasFYD/CS528-HWs
Full logs in CS528-HW2/results/: local_full_run.txt, cloud_shell_resume_run.txt, and vm_full_run.txt.
Billing delay reference: https://docs.cloud.google.com/billing/docs/how-to/resolve-issues